# Arquitectura del Sistema — SIA · IES N° 11

**Sistema de Asistencia Inteligente (Python Edition)**
Instituto de Educación Superior N° 11 · Tech & Innovation Team
Versión del documento: noviembre de 2026

## 1. Visión general

SIA es una aplicación web en **Streamlit** organizada en cuatro capas:

| Capa | Carpeta | Responsabilidad |
|---|---|---|
| Presentación | views/ | Pantallas, componentes visuales, escáner QR, PWA y estilos. No accede a la base directamente. |
| Dominio | models/ | Clases de negocio (POO): personas, materias, asistencia y predictor de IA. |
| Infraestructura | core/ | Base de datos, seguridad, validaciones, fechas, reportes PDF/Excel y Google Drive. |
| Utilidades | utils/ | Generación y lectura de códigos QR. |

Flujo de una acción típica: la vista (views/) recibe la interacción, llama a un método del modelo (models/), que valida y persiste mediante core/database.py; la vista vuelve a dibujarse con el resultado.

## 2. Estructura del repositorio

```
sia-python-edition/
├── app.py                    Punto de entrada: página, sesión y ruteo por rol
├── seed_init.py              Verifica e inicializa la base de datos
├── requirements.txt          Dependencias de Python
├── packages.txt              Paquetes del sistema para Streamlit Cloud
├── README.md                 Guía de uso y despliegue
├── .streamlit/
│   ├── config.toml           Tema y configuración del servidor
│   └── secrets.toml.example  Plantilla de Secrets (SIA_SECRET, Drive, etc.)
├── .devcontainer/            Entorno de desarrollo para GitHub Codespaces
├── assets/
│   ├── logo_ies11.png        Logo del IES N° 11 (encabezado y barra lateral)
│   ├── logo_tech.png         Logo de Tech & Innovation Team (pie de página)
│   └── LEEME.md              Cómo reemplazar los logos
├── static/                   Servido en /app/static/ (PWA)
│   ├── manifest.json         Manifiesto de la app instalable
│   └── icon-*.png            Íconos (192, 512, adaptable y Apple)
├── core/
│   ├── database.py           Esquema SQLite, conexiones y migraciones
│   ├── security.py           Hash de contraseñas, firmas HMAC, lectura de Secrets
│   ├── validaciones.py       Opciones de los selectores y validaciones
│   ├── tiempo.py             Fecha y hora en la zona horaria institucional
│   ├── reportes.py           PDF de cierre de clase y planillas Excel
│   └── drive.py              Subida de archivos a Google Drive
├── models/
│   ├── persona.py            Persona (abstracta), Docente, Estudiante y RepositorioPersonas
│   ├── materia.py            Materia: cursada, inscripciones, clases y programa PDF
```

```
│   ├── asistencia.py          RegistroAsistencia: tokens QR, PIN, marcado idempotente
│   └── predictor_ia.py        PredictorRiesgoIA: Random Forest y análisis por reglas
├── utils/
│   └── qr.py                  Genera y lee códigos QR (OpenCV / pyzbar)
├── views/
│   ├── auth.py                Acceso, registro, sesión con expiración, cambio de contraseña
│   ├── docente.py             Panel docente (Materias, Clase en vivo, Lector QR, Historial)
│   ├── alumno.py              Panel estudiante (Presente, Unirme, Mi historial, Credencial)
│   ├── escaner.py             Escáner QR en vivo (st.components.v2 + MediaDevices)
│   ├── exportar.py            Botones de CSV, Excel y Google Drive
│   ├── pwa.py                 PWA y ocultamiento de elementos de Streamlit Cloud
│   └── styles.py              Paleta, tipografía, modo oscuro, componentes HTML responsivos
├── scripts/
│   └── drive_autorizar.py     Autoriza Google Drive (se ejecuta una vez)
└── docs/                      Informes del sistema (.md y .pdf) y su generador
```

# 3. Módulos principales

## 3.1 app.py

1. Descarta módulos del proyecto desactualizados en memoria (evita errores de importación tras un redespliegue en Streamlit Cloud).
2. Configura la página e inicializa la base (una vez por proceso).
3. Captura enlaces QR abiertos con la cámara del celular (?asistencia= o ?inscribir=), los valida en el momento y los guarda como acción pendiente.
4. Inyecta estilos y la configuración PWA, valida la sesión y deriva al panel según el rol.

## 3.2 Modelo de datos (SQLite)

| Tabla | Contenido |
|---|---|
| usuarios | DNI, nombre, apellido, email, rol, hash y sal de la contraseña, legajo o departamento |
| materias | Nombre, descripción, curso, comisión, turno, días, hora de inicio y fin, texto de horario, umbral, código de clase, docente |
| materia_programas | Programa en PDF por materia (tabla aparte para no cargar el archivo al listar) |
| inscripciones | Alumno y materia, fecha de inscripción, resultado final |
| clases | Una por materia y fecha: tema, abierta o cerrada |
| asistencias | Una por alumno y clase (UNIQUE): estado, método, quién y cuándo registró |
| clase_reportes | PDF generado al cerrar cada clase |
| intentos_login | Intentos de acceso para el bloqueo temporal |
| config | Versión del esquema y clave HMAC generada si no se configuró SIA_SECRET |

Las migraciones son automáticas: init_db() crea las tablas faltantes y agrega columnas nuevas a bases existentes sin perder datos (versión de esquema actual: 3).

## 3.3 Asistencia por QR

| Código | Formato | Quién lo escanea |
|---|---|---|
| QR de clase (dinámico) | SIA:C:<clase>:<ventana>:<firma> o enlace ?asistencia= | Estudiante |
| PIN | 6 dígitos derivados de la misma ventana | Estudiante (sin cámara) |
| Credencial | SIA:STUDENT:<dni>:<firma> (fija) | Docente |
| Inscripción | SIA:ENROLL:<código>:<firma> o enlace ?inscribir= | Estudiante |

Las firmas son HMAC-SHA256 con SIA_SECRET. La ventana del QR dinámico dura 30 segundos y se acepta la anterior. El marcado es idempotente gracias a la restricción UNIQUE(clase_id, estudiante_id).

### 3.4 Escáner en vivo (views/escaner.py)

Componente st.components.v2 (HTML, CSS y JavaScript que corren en la página):

1. getUserMedia abre la cámara elegida, la recordada o la trasera; enumerateDevices arma el selector.
2. Cada 150 ms decodifica un cuadro con BarcodeDetector nativo o, si el navegador no lo trae (Chrome y Edge en Windows), con jsQR.
3. Al leer un código, setTriggerValue lo envía a Python, que procesa la lectura en una única recarga. El mismo código no se reenvía durante 8 segundos.
4. Ante errores (permiso denegado, cámara en uso, sin cámara, sitio sin https) muestra un mensaje en español y el botón Reintentar.

### 3.5 Reportes y Google Drive

- core/reportes.py: PDF con fpdf2 y fuentes base (funciona igual en Windows y Linux); Excel con openpyxl.
- core/drive.py: subida multipart a Drive API v3 con google-auth, permiso drive.file y soporte de Unidades compartidas.

### 3.6 Interfaz responsiva

views/styles.py define los componentes HTML propios: tarjetas, indicadores, tabla_html (se vuelve tarjetas en pantallas angostas), grilla_historial (íconos que pasan de línea) y banner_riesgo. Las pestañas se acomodan en varias filas. Resultado: ninguna vista tiene scroll horizontal.

## 4. Configuración (Secrets)

| Clave | Uso |
|---|---|
| SIA_SECRET | Clave HMAC de QR y credenciales. Obligatoria en producción. |
| SIA_DOCENTE_CODE | Código institucional para el alta de docentes (opcional). |
| SIA_PUBLIC_URL | URL pública para que los QR sean enlaces (opcional; se detecta sola). |
| SIA_TZ / SIA_SESSION_MINUTES | Zona horaria y minutos de inactividad (opcionales). |
| SIA_DRIVE_FOLDER_ID | Carpeta de destino en Google Drive. |

| Clave | Uso |
|---|---|
| SIA_DRIVE_SERVICE_ACCOUNT | JSON de la cuenta de servicio (modo 1). |
| SIA_DRIVE_CLIENT_ID, SIA_DRIVE_CLIENT_SECRET, SIA_DRIVE_REFRESH_TOKEN | Cuenta de Google autorizada (modo 2). |

### Google Drive

**Modo 1 — Cuenta de servicio (Google Workspace con Unidades compartidas):** crear una cuenta de servicio en Google Cloud con la API de Drive habilitada, descargar su JSON, compartir con su email una carpeta dentro de una Unidad compartida (rol Editor) y cargar SIA_DRIVE_SERVICE_ACCOUNT y SIA_DRIVE_FOLDER_ID.

**Modo 2 — Cuenta de Google autorizada (sirve con cuentas @gmail.com):** crear un "ID de cliente de OAuth" de tipo App de escritorio, ejecutar una vez python scripts/drive_autorizar.py client_secret.json y pegar en Secrets los valores que muestra, más SIA_DRIVE_FOLDER_ID (la parte final de la URL de la carpeta).

Mientras Drive no esté configurado, los botones "Enviar a Google Drive" aparecen deshabilitados con una explicación.

## 5. Despliegue

1. Subir el repositorio a GitHub y crear la app en Streamlit Community Cloud apuntando a app.py.
2. Cargar los Secrets (al menos SIA_SECRET).
3. packages.txt instala libzbar0; requirements.txt, las dependencias de Python.
4. Tras cada actualización, si algo se ve desactualizado: Manage app, menú, Reboot app.

**Persistencia:** el disco de Streamlit Cloud es efímero. Para uso institucional prolongado, conviene un servidor con disco persistente (SIA_DB_PATH) o copias periódicas de database.db.

## 6. Verificación

| Control | Herramienta |
|---|---|
| Sintaxis de todos los módulos | python -m compileall |
| Estilo y errores comunes | ruff check --select E,F,W,B |
| Arranque de todas las vistas (3 roles, modo claro y oscuro) | streamlit.testing.v1.AppTest |
| Escáner, cierre de clase y scroll horizontal | Navegador con cámara simulada y medición del DOM en 320, 375 y 1280 px |

## 7. Generación de estos documentos

Los PDF de docs/ se generan a partir de los .md:

```
pip install markdown fpdf2
python docs/generar_pdfs.py
```